

Calculated Relations — Design and Roadmap

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Background	2
2. Problem Statement.....	3
3. Goals	4
4. Non-Goals	5
5. Design	6
5.1. D1. Materialization Modes	6
5.2. D2. Dependency Validation and Loud Staleness	7
5.3. D3. Declarative Hierarchy/List Edges.....	8
5.4. D4. Compute Guards	8
5.5. D5. Provenance Off the Hot Path.....	9
5.6. D6. Bulk Recompute CLI	10
5.7. D7. Metrics	10
5.8. D8. Provider Test Kit	11
6. Roadmap.....	12
6.1. Sequencing Rationale	12
7. Backwards Compatibility	13
8. Open Questions.....	14
9. Out of Scope (For Now)	15

Status: Implemented on [feature/ontology-enhancements](#) against [1.4.0-SNAPSHOT](#).

Owners: Ontology team. **Scope:** [quantum-ontology-core](#), [quantum-ontology-mongo](#), [quantum-ontology-policy-bridge](#), [quantum-cli](#), [quantum-docs](#).



Implementation status (initial pass): All eight design items have landed. Per-item notes appear inline in the design and the roadmap table below. In short:

- M1 (Safety Net): D7 metrics, D4 guards, D2 [@DependsOn](#) + validator, D8 test kit — all complete with unit tests in [quantum-ontology-core](#).
- M2 (Operability): D6 bulk recompute service + admin REST; D5 provenance side-collection storage and migration runner. The CLI shim and the upsert-path rewrite are explicitly deferred (see notes).
- M3 (Performance / Authoring): D1 materialization modes (EAGER/LAZY/ONDEMAND) with cache + read facade + write-hook honoring; D3 declarative [computed](#): YAML block + [DeclarativeComputedEdgeProvider](#).

Total new test coverage in [quantum-ontology-core](#): 72 tests (was 53).

This document captures a forward-looking design for the **calculated relations** subsystem (computed/derived ontology edges). It collects the gaps observed in the current implementation, proposes targeted enhancements, and sequences them into a phased roadmap.

For a usage-level introduction to ontology relationships, see [user-guide/ontology.adoc](#) (including the Territory → Location computed-edge example under *Custom Edge Providers*). A longer narrative guide also lives at [quantum-docs/src/docs/implementing-territory-location-edges.md](#).

Chapter 1. Background

The framework distinguishes three ways edges enter `ontology_edges`:

Kind	<code>inferred</code>	<code>derived</code>	Source
Explicit	false	false	<code>@OntologyProperty</code> annotation or YAML triple
Inferred	true	false	<code>ForwardChainingReasoner</code> (transitive, symmetric, subPropertyOf, chains)
Derived (calculated)	false	true	<code>ComputedEdgeProvider</code> SPI implementations

The flags live on `EdgeRecord` (`quantum-ontology-core/.../EdgeRecord.java`).

Calculated edges are produced by CDI beans extending `ComputedEdgeProvider<S>` (or its specialization `HierarchyListEdgeProvider<S, H, T>`), discovered via `ComputedEdgeRegistry`, and triggered post-persist by `OntologyWriteHook`. Each edge carries a `ComputedEdgeProvenance` record (`providerId, hierarchyPath, resolvedLists, computedAt`).

Chapter 2. Problem Statement

Calculated relations work, but the current implementation has rough edges that will become painful as the number of providers and the size of materialized edge sets grow:

1. **Eager-only materialization.** Every relevant entity write recomputes and rewrites the full edge set for affected sources. There is no compute-on-read or cached tier.
2. **Silent staleness.** Providers that declare `getDependencyTypes()` but leave `getAffectedSourceIds()` returning the default empty set will quietly skip invalidation. There is no startup-time validation and no runtime warning.
3. **No declarative path for the common shape.** The "source assigned to hierarchy node, node carries list of targets" pattern is implemented by hand in every provider, even though `HierarchyListEdgeProvider` already shapes the structure.
4. **No automatic cycle/depth guards in compute.** The reasoner (`ForwardChainingReasoner`) guards transitive expansion with `MAX_TRANSITIVE_STEP_VISITS = 10_000` (visit budget, not a 10-iteration cap), but provider-side compute historically relied on the implementer to terminate (addressed by D4).
5. **Provenance bloat.** Per-edge provenance maps are stored inline on every `EdgeRecord`, inflating the working set for high-fanout providers.
6. **No bulk recompute / reconciliation tool.** When provider logic changes, there is no first-class way to reprocess existing sources.
7. **No metrics surface.** Per-provider invocation count, duration, and add/remove deltas are not exposed.
8. **Heavy test setup.** Provider unit tests require Mongo and full repo wiring.

Chapter 3. Goals

- Reduce write-amplification for high-fanout providers without changing the read API.
- Make staleness bugs hard to introduce — fail loud, not silent.
- Lower the cost of authoring a new calculated relation for the common case to roughly "fill out a YAML block".
- Give operators visibility (metrics) and a recovery path (recompute CLI).
- Keep the existing `ComputedEdgeProvider` contract as the escape hatch; do not break callers.

Chapter 4. Non-Goals

- A full SPARQL/OWL query layer. The expression DSL stays narrow.
- Replacing the reasoner. Inferred edges remain the reasoner's domain.
- Cross-realm or cross-tenant edge sharing.

Chapter 5. Design

5.1. D1. Materialization Modes

Status: implemented



- `MaterializationMode` enum at `quantum-ontology-core/.../core/MaterializationMode.java`.
- `ComputedEdgeProvider#getMaterializationMode()` (default `EAGER`) and `getCacheTtlSeconds()` hooks.
- `ComputedEdgeCache` (per-process TTL'd map) at `quantum-ontology-core/.../core/ComputedEdgeCache.java`.
- `OntologyWriteHook` skips non-`EAGER` providers at write time.
- `ComputedEdgeReader` (in `quantum-ontology-mongo`) is the read-side facade for source-keyed **and** destination-keyed lookups used by policy/`hasEdge` rewrites; `ComputedEdgeRecomputeHandler` punches the cache for `LAZY` and no-ops for `ONDEMAND` on dependency changes.

Add a `MaterializationMode` enum returned by a new `ComputedEdgeProvider#getMaterializationMode()` (default `EAGER`):

```
public enum MaterializationMode {  
    EAGER,    // current behavior: write through to ontology_edges  
    LAZY,     // compute on read; cache with TTL  
    ONDEMAND // never store; always compute via the repo facade  
}
```

`ComputedEdgeReader` is the mode-aware facade. Callers that need correctness across all modes must use it (or APIs wired through it) rather than `OntologyEdgeRepo` alone:

- **Source-keyed reads** (`edgesFor` / `dstIdsBySrc`): for `LAZY/ONDEMAND`, short-circuit to the provider (with TTL cache for `LAZY`) instead of the collection.
- **Destination-keyed reads** (`srcIdsByDst`, used by `hasEdge` / `ListQueryRewriter`, MCP incoming-graph tools, and similar): for `LAZY/ONDEMAND`, enumerate candidate sources via `SourceEntityEnumerator`, compute provider edges, and union with any `EAGER` rows still in `ontology_edges`. Without an enumerator, inverse results fall back to store-only and log a warning (incomplete for pure `LAZY` providers).
- `OntologyEdgeRepo` remains the pure store. Direct repo reads do **not** see unmaterialized `LAZY/ONDEMAND` edges.

The `LAZY` cache lives in a `ComputedEdgeCache` (per-process, TTL- configurable per provider). Cache invalidation reuses the existing `getAffectedSourceIds` machinery.

Acceptance:

- An existing provider can opt into **LAZY** by overriding one method.
- Source-keyed and destination-keyed reads return the same edges as the **EAGER** path when a **SourceEntityEnumerator** (and source loader) is registered for the provider's source type.
- Writes that previously fanned out N edges no longer write any rows for that provider.
- Inverse enumeration is bounded (visit budget) and may be expensive; prefer **EAGER** when destination-side policy filters are hot paths.

5.2. D2. Dependency Validation and Loud Staleness

Status: implemented



- `@DependsOn(type, via, expand)` (repeatable) at `quantum-ontology-core/.../annotations/DependsOn.java`.
- `ComputedEdgeProvider#getDependencyTypes()` derives its default from `@DependsOn` declarations on the class hierarchy.
- `ComputedEdgeStalenessValidator` is pure logic in core, unit-tested without CDI.
- `ComputedEdgeProviderAutoDiscovery` runs the validator at startup, emits **WARN** logs, and bumps `OntologyMetrics.recordStalenessRisk` per unhelped (provider, dependency-type) pair.

Add startup-time validation in `ComputedEdgeRegistry`:

1. For each registered provider, walk `getDependencyTypes()`.
2. For each dependency type, check whether `getAffectedSourceIds(...)` for an arbitrary id returns the default empty set **and** the method has not been overridden (detected via reflection on the declaring class).
3. Log a **WARN** and emit a Micrometer counter `ontology.provider.staleness_risk` tagged by `providerId` and `dependencyType`.

Add an optional `@DependsOn` annotation that declaratively expresses the inverse-edge query for the common case:

```
@DependsOn(type = Territory.class, via = "assignedTerritories", expand = ANCESTORS)
@DependsOn(type = LocationList.class, via = "staticDynamicList")
public class AssociateCanSeeLocationProvider extends HierarchyListEdgeProvider<...> {
}
```

The registry derives `getAffectedSourceIds` from the annotation if the method has not been explicitly overridden.

Acceptance: removing the override on a provider with `@DependsOn` keeps invalidation correct; removing both surfaces a warning at startup.

5.3. D3. Declarative Hierarchy/List Edges

Status: implemented



- YAML `computed:` block parsed by `YamlOntologyLoader.loadComputedSpecs()` into `ComputedEdgeSpec` records.
- `DeclarativeRuntime<S, H, T>` SPI for application-supplied entity loading / hierarchy walk / list resolution.
- `DeclarativeComputedEdgeProvider` extends `HierarchyListEdgeProvider`, so the cycle/depth/target guards from D4 apply automatically.
- End-to-end test: `DeclarativeComputedEdgeTest` loads a YAML spec and produces the expected target set against in-memory fakes.
- `dependsOn:` entries are surfaced via `declaredDependencyTypeNames()`; applications resolve type strings to classes when registering a runtime.

Extend the YAML ontology schema to allow `computed:` definitions for the hierarchy+list pattern:

```
computed:
- id: canSeeLocation
  domain: Associate
  range: Location
  via:
    hierarchy:
      node: Territory
      from: assignedTerritories      # field on Associate
      expand: descendants             # descendants | ancestors | none
    list:
      field: locationLists           # field on Territory
      mode: auto                     # auto | static | dynamic
  dependsOn:
    - type: Territory
    - type: LocationList
```

A `YamlComputedEdgeProvider` implementation reads these blocks at boot and registers a synthetic provider per entry. Hand-rolled providers remain the escape hatch for everything that does not fit this shape.

Acceptance: the territory/location reference example can be expressed entirely in YAML and produces edges identical to the hand-rolled provider under the existing integration tests.

5.4. D4. Compute Guards

Status: implemented



- `ComputationContext` now tracks `visitedNodeIds`, configurable `maxHierarchyDepth` (default 64) and `maxComputedTargets` (default 50_000), and a list of guard trips.
- `HierarchyListEdgeProvider` short-circuits on cycle / depth / target hits and

records the trip. (Note: with the existing flat-descendants API, `maxHierarchyDepth` is interpreted as max hierarchy nodes processed, not tree depth — see `HierarchyListGuardsTest`.)

- Trips flow into `OntologyMetrics.recordGuardTrip` per provider via the static metrics hook on `ComputedEdgeProvider`.

Augment `ComputationContext` with:

- `Set<String> visitedNodeIds` — automatically populated by `HierarchyListEdgeProvider` during BFS; revisits short-circuit.
- `int maxHierarchyDepth` (configurable, default 64).
- `int maxComputedTargets` (configurable, default 50_000).

When a guard trips, the provider returns the partial result, logs a `WARN`, and increments a counter tagged by provider. Bare `ComputedEdgeProvider` implementations get the same context and can opt in.

Acceptance: a synthetic test with a cyclic Territory hierarchy completes without stack overflow and emits the expected warning counter.

5.5. D5. Provenance Off the Hot Path

Status: storage + migration implemented; upsert wiring deferred



- `OntologyEdgeProvenanceDoc` (collection `ontology_edge_provenance`), `OntologyEdgeProvenanceRepo`, and `ProvenanceStore` strategy.
- Feature flag `quantum.ontology.provenance.split` (default false). With the flag off, behavior is unchanged.
- `ProvenanceMigrationService.migrate(realm, dryRun)` is idempotent and exposed at `POST /ontology/admin/provenance/migrate`.
- **Deferred:** rewriting `OntologyMaterializer` and `OntologyEdgeRepo` upsert paths to consult `ProvenanceStore` rather than setting `prov` directly. Doing so without the integration-test environment risked regressions; the storage and migration are the correct foundation and can be wired in with full IT coverage in a follow-up.

Move per-edge provenance out of `EdgeRecord.prov` into a sibling collection `ontology_edge_provenance` keyed by edge `_id`. `EdgeRecord` retains a lightweight `provRef` (boolean: provenance available) and `providerId` for filtering.

Graph REST endpoints accept `?withProv=true` to join provenance into the response. The policy bridge does not need provenance, so its hot path shrinks proportionally.

Acceptance: an edge query with `withProv=false` returns documents whose average size is reduced by the previously-inlined provenance bytes; with `withProv=true` the response is byte-equivalent (modulo ordering) to today.

Migration: a one-shot job copies existing `prov` maps to the side collection and clears the inline field.

5.6. D6. Bulk Recompute CLI

Status: service implemented; REST endpoint exposed; CLI shim deferred



- **BulkRecomputeService** walks all source entities for a provider via a registered **SourceEntityEnumerator** SPI, recomputes target sets, diffs against persisted edges, and either reports deltas (**dryRun=true**) or re-applies via the existing **ComputedEdgeRecomputeHandler**.
- Exposed at **POST /ontology/admin/recompute** with the original request/response shape.
- **GET /ontology/admin/metrics** exposes the **OntologyMetrics** snapshot.
- **Deferred:** the picocli verb. **quantum-cli** already binds **@TopCommand** to **ServiceTokenCLI**; quarkus-picocli only allows one top command, so adding a sibling would either restructure the existing CLI invocation or require a parallel jar. The REST endpoint is the canonical entry point; a thin CLI wrapper that posts to it can land separately.

Add a **quantum-cli** verb:

```
quantum ontology recompute \  
  --provider AssociateCanSeeLocationProvider \  
  --realm acme \  
  [--source-id <id>] [--dry-run] [--batch-size 500]
```

It iterates source entities (filtered to **--source-id** if given), runs the provider's **computeTargets**, diffs against current materialized edges, and applies (or reports, with **--dry-run**) the deltas. Output: per-source add/remove counts and a final summary.

Acceptance: running with **--dry-run** after a no-op deploy reports zero deltas; running after a deliberate provider-logic change reports the expected diff.

5.7. D7. Metrics

Status: implemented (in-process; Micrometer bridge follow-up)



- **OntologyMetrics** is a dependency-free **@ApplicationScoped** bean in **quantum-ontology-core**. Per-provider counters: invocations, total / avg / max nanos, targets produced, edges added, edges removed, errors, guard trips, staleness-risk-by-dependency.
- Wired in **OntologyWriteHook** (timing per provider invocation) and **OntologyMaterializer** (added/removed deltas via **prov.providerId** inspection).
- Snapshot exposed at **GET /ontology/admin/metrics**.
- **Follow-up:** a Micrometer bridge (subscribe to **OntologyMetrics** snapshots and emit gauges/counters) is intentionally separate so **quantum-ontology-core** stays free of the Micrometer dependency.

Wrap `ComputedEdgeProvider#edges(...)` in a Micrometer timer and counters, tagged by `providerId`:

- `ontology.provider.invocations` — counter
- `ontology.provider.duration` — timer
- `ontology.provider.targets_produced` — distribution summary
- `ontology.provider.edges_added`, `ontology.provider.edges_removed` — counters (emitted from the materializer diff path)

Surfaced through the existing Quarkus Micrometer registry.

Acceptance: `/q/metrics` exposes the counters with non-zero values after the integration test run.

5.8. D8. Provider Test Kit

Status: implemented



- `com.e2eq.ontology.testkit.InMemoryHierarchy<H, T>` builds a fluent fake hierarchy with parent/child links and per-node list items; its `descendants(nodeId)` matches the `HierarchyListEdgeProvider` flat-descendants contract.
- `com.e2eq.ontology.testkit.ComputedEdgeProviderHarness` invokes any `ComputedEdgeProvider` against a fake `DataDomainInfo` and returns a `Result` with `targetIds()` and `providerIds()` helpers.
- Sample test (`TestKitSampleTest`) declares a hierarchy + provider in ~80 lines with no Mongo and no Quarkus boot.
- Drive-by fix: `extractId`, `extractNodeId`, and `extractNodeRefName` now `setAccessible(true)` on the reflected `getId/getRefName` methods, so providers work against package-private fixtures and reflection-loaded classes.

Add `quantum-ontology-core/src/test-fixtures` (or a `test-jar`) module with:

- `InMemoryHierarchyRepo<H>` — accepts a tree literal in tests.
- `InMemoryListResolver<T>` — registered (`listId` → `items`) map.
- `ComputedEdgeProviderHarness` — wires a provider against the fakes and asserts target sets and provenance.

The provider unit test for `AssociateCanSeeLocationProvider` is rewritten against the harness; the existing `TerritoryLocationPermissionIT` remains the integration coverage.

Acceptance: the provider unit tests no longer depend on Mongo or Quarkus boot; they run in <1s.

Chapter 6. Roadmap

The roadmap is organized into three sequential milestones inside the **1.4.0-SNAPSHOT** line. Each milestone is independently shippable.

Milestone	Theme	Items	Risk	Target	Status
M1 — Safety Net	Make staleness loud, give us metrics and a test kit before we change behavior.	• D2 — Dependency validation and <code>@DependsOn</code> • D4 — Compute guards • D7 — Metrics • D8 — Provider Test Kit	Low. Additive only; no on-disk format change.	1.4.0-SNAPSHOT early	Done
M2 — Operability	Recovery path and provenance footprint.	• D6 — Bulk recompute (REST; CLI shim deferred) • D5 — Provenance side collection + migration (upsert wiring deferred)	Medium. Migration touches existing data; gated by feature flag <code>quantum.ontology.provenance.split</code> (default off until validated).	1.4.0-SNAPSHOT mid	Done (with documented deferrals)
M3 — Performance and Authoring	Reduce write amplification and lower the cost of new providers.	• D1 — Materialization modes (EAGER, LAZY, ONDEMAND) • D3 — Declarative computed edges in YAML	Medium-high. New code paths in the read layer; YAML schema additions.	1.4.0-SNAPSHOT late / branch handoff	Done

6.1. Sequencing Rationale

- M1 ships first because its items are observability and validation. They surface latent bugs that M2/M3 might otherwise hide.
- M2 ships before M3 because D5 (provenance split) is a prerequisite for honest cost comparisons of D1 (LAZY/ONDEMAND).
- D3 is intentionally last; we want to validate the declarative shape against real providers ported in M2/M3, not against speculative ones.

Chapter 7. Backwards Compatibility

- All `ComputedEdgeProvider` subclasses continue to compile and run unchanged.
- The new `getMaterializationMode()` defaults to `EAGER`.
- The new `@DependsOn` annotation is opt-in; missing annotations behave as today.
- The provenance split is gated behind a feature flag; with the flag off, `EdgeRecord.prov` continues to be populated inline.

Chapter 8. Open Questions

1. Should **LAZY** results be cached per (source, predicate) or per source? Per-source is simpler but invalidates more aggressively.
2. Where does **@DependsOn**-derived invalidation actually run the inverse query — synchronously in the write hook, or via a lightweight worker?
3. For the YAML DSL, do we surface filters on the list step (**list.filter: "active = true"**), or push that to the existing static/dynamic list machinery?
4. Should the recompute CLI emit an idempotent ledger so re-runs converge, or is the diff-and-apply path sufficient?
5. Should inverse LAZY scans maintain a secondary index (dst → src) instead of enumerating sources on each **srcIdsByDst** call?

These are deferred to design discussion as follow-ups after the initial modes land.

Chapter 9. Out of Scope (For Now)

- A general expression language for arbitrary computed edges (vs. just the hierarchy+list shape). Tracked for a future major.
- Cross-provider memoization (e.g. two providers walking the same Territory tree share work). Plausible after M3 but not in this roadmap.
- GraphQL-style on-the-fly traversal. Out of scope; use the existing graph REST endpoints.